

BatchNorm2d#

<https://pytorch.org/docs/stable/generated/torch.nn.modules.batchnorm.BatchNorm2d.html>

Description:

Applies Batch Normalization over a 4D input. 4D is a mini-batch of 2D inputs with additional channel dimension. Method described in the paper [Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift](#). The mean and standard-deviation are calculated per-dimension over the mini-batches and γ and β are learnable parameter vectors of size C (where C is the input size). By default, the elements of γ are set to 1 and the elements of β are set to 0. At train time in the forward pass, the standard-deviation is calculated via the biased estimator, equivalent to `torch.var(input, unbiased=False)`. However, the value stored in the moving average of the standard-deviation is calculated via the unbiased estimator, equivalent to `torch.var(input, unbiased=True)`. Also by default, during training this layer keeps running estimates of its computed mean and variance, which are then used for normalization during evaluation. The running estimates are kept with a default momentum of 0.1. If `track_running_stats` is set to `False`, this layer then does not keep running estimates, and batch statistics are instead used during evaluation time as we

Function Signature

```
class torch.nn.modules.batchnorm.BatchNorm2d(num_features,  
eps=1e-05, momentum=0.1, affine=True,  
track_running_stats=True, device=None, dtype=None)[source]#
```

Parameters

Shape:

Code Examples

```
>>> # With Learnable Parameters  
>>> m = nn.BatchNorm2d(100)  
>>> # Without Learnable Parameters  
>>> m = nn.BatchNorm2d(100, affine=False)  
>>> input = torch.randn(20, 100, 35, 45)  
>>> output = m(input)
```

Notes & Warnings

NOTE

This momentum argument is different from one used in optimizer classes and the conventional notion of momentum. Mathematically, the update rule for running statistics here is $x^{\text{new}} = (1 - \text{momentum}) \times x^{\text{old}} + \text{momentum} \times \hat{x}_{\text{new}}$ where x^{old} is the estimated statistic and $\hat{x}_{\text{new}}$ is the new observed value.

Detailed Documentation

Documentation

Applies Batch Normalization over a 4D input.

4D is a mini-batch of 2D inputs with additional channel dimension. Method described in the paper [Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift](#).

The mean and standard-deviation are calculated per-dimension over the mini-batches and γ and β are learnable parameter vectors of size C (where C is the input size). By default, the elements of γ are set to 1 and the elements of β are set to 0. At train time in the forward pass, the standard-deviation is calculated via the biased estimator, equivalent to `torch.var(input, unbiased=False)`. However, the value stored in the moving average of the standard-deviation is

calculated via the unbiased estimator, equivalent to `torch.var(input, unbiased=True)`.

Also by default, during training this layer keeps running estimates of its computed mean and variance, which are then used for normalization during evaluation. The running estimates are kept with a default momentum of 0.1.

If `track_running_stats` is set to `False`, this layer then does not keep running estimates, and batch statistics are instead used during evaluation time as well.

This momentum argument is different from one used in optimizer classes and the conventional notion of momentum. Mathematically, the update rule for running statistics here is $x^{\text{new}} = (1 - \text{momentum}) \times x^{\text{old}} + \text{momentum} \times x_{\text{txt}}$, where x^{old} is the estimated statistic and x_{txt} is the new observed value.

Because the Batch Normalization is done over the C dimension, computing statistics on (N, H, W) slices, it's common terminology to call this Spatial Batch Normalization.

`num_features` (int) – CCC from an expected input of size (N,C,H,W)(N, C, H, W)(N,C,H,W)

`eps` (float) – a value added to the denominator for numerical stability. Default: 1e-5

`momentum` (Optional[float]) – the value used for the `running_mean` and `running_var` computation. Can be set to `None` for cumulative moving average (i.e. simple average). Default: 0.1

`affine` (bool) – a boolean value that when set to `True`, this module has learnable affine parameters. Default: `True`

`track_running_stats` (bool) – a boolean value that when set to `True`, this module tracks

the running mean and variance, and when set to False, this module does not track such statistics, and initializes statistics buffers `running_mean` and `running_var` as None. When these buffers are None, this module always uses batch statistics. in both training and eval modes. Default: True

Input: (N,C,H,W)(N, C, H, W)(N,C,H,W)

Output: (N,C,H,W)(N, C, H, W)(N,C,H,W) (same shape as input)